18.04.25

“The objective for this week is to review the basics of Flatland so that everyone is on the same page, and so that we all have a good foundation upon which to build our encodings.”

So, there’s not much to be done.

“Come up with a team name”

“Railroad Gnomes”

“and begin working on Assignment #1”

Let’s see what’s in the link. It seems that there might be an exact task:

“The purpose of this exercise is to get your brains thinking about the logic and mechanics behind navigating several trains around a Flatland environment.”

Yes, I was right: it is about doing something very exact. Perhaps, I'll have to model the environment.

But that’s all - there is no task I see, only a submission button.

Ah no - clicking on it led me to the task itself:

1. “*Draw a small environment.*

It can be done on paper, on a tablet, or on the computer. Focus on keeping it small, but try to make use of a variety of tracks. Make sure to consider where the trains start and where their target stations are.

2. *Represent the environment with facts.*

Using the fact formats presented in the slides, provide an .lp file that accurately represents the environment.

3. *Encode a pathfinding solution.*

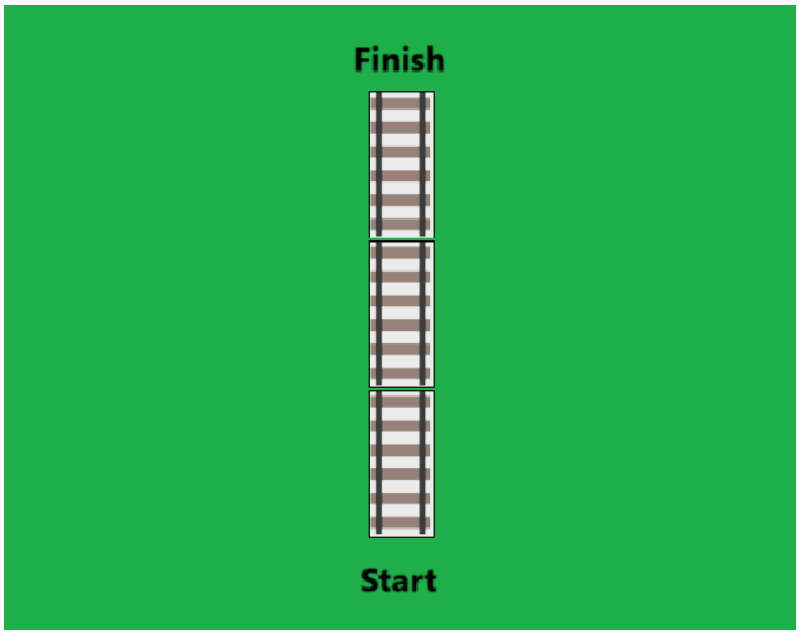
Write an encoding that, given your environment representation (or one like it), can move the trains from start to finish without colliding.”

That’s already a lot! Especially the third point. Let’s then come up with the simplest possible train “network”.
First, I need to explore the available track segments:

https://moodle2.uni-potsdam.de/pluginfile.php/3253285/mod_resource/content/1/Track%20Types.pdf

Network 1:

The simplest possible track is a straight line. Let's compile a network of a few straight tracks:



The picture where I can see track types assigns some weird numbers to them. For example, a no-track "of any angle" has index 0.

Straight track (all tracks that don't have turns are called "straight" in the legend, but here I mean the actually straight track), has index 32800 (south-north) or 1025 (east-west).

Then, in this simple network the only type of track is straight south-north, which is 32800.

What's next? Next, I need to describe this simple environment in code. What are the elements? First, it makes sense to describe it in words as closely as possible:

There are 3 sections. All sections are of the same type: 32800. (I've also marked "Start" and "Finish" - but does it make sense according to the task? To me, intuitively, now it seems necessary - to locate the train correctly at the beginning.)

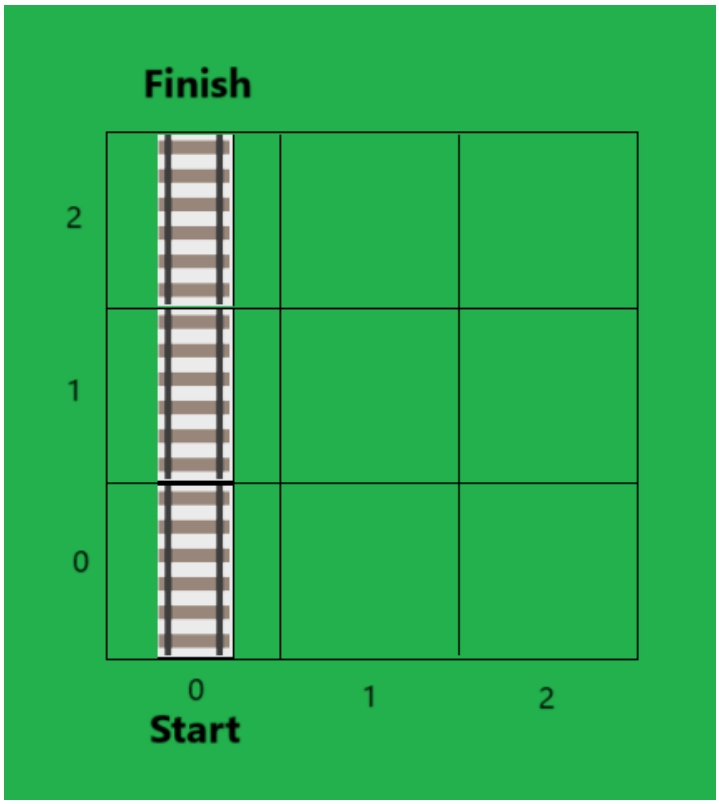
Intuition: create an atom "segment type" - no, segment! And in the segment I'll be able to locate it on a 2d grid. The 2d grid can be put either in a tuple or in two separate variables:

```
segment(type, (x, y)).
```

Or

```
segment(type, x, y).
```

The first one seems more appealing, since the coordinates are logically grouped and separated from the “type” object.



This is an easily-codable representation of the railroad network - as a simple grid world. The difference, however, is that while coordinates span the whole rectangle, the tracks are laid only in certain locations, tying the train movement to it.

What would it look like in Python?

As a 2d list / numpy array:

```
network_1 = [[0, 0, 0]
             [0, 0, 0]
             [0, 0, 0]]
```

And because we have different track types, we can denote both the presence of the route and the type of the segment simultaneously - by replacing 0 in the respective location with the segment type code:

```
network_1 = [[32800, 0, 0]
             [32800, 0, 0]
             [32800, 0, 0]]
```

So, the number in the array bears a double meaning:

- There is a path if the number is different from 0;
- The type of the path is the number;

What’s next?

Next, I need to somehow represent the train.

Intuition: the train must “see” the type of the track it is on. For example, if the train “sees” that the track type is 32800, the only possible movement for it is:

- One cell to the north if travelling north;
- One cell to the south if travelling south.

Since I'm aiming for the simplest case now, the best will be to set one possible direction of movement: from south to north.

So, if the train "sees" that the segment type is 32800, it moves one cell north and reads the next segment type, "deciding" on the next move (there might be a binary choice for more complex cases in the future, but for now there is no choice - just action defined by the type of the segment).

"you may not move backward"

What is "movement"?

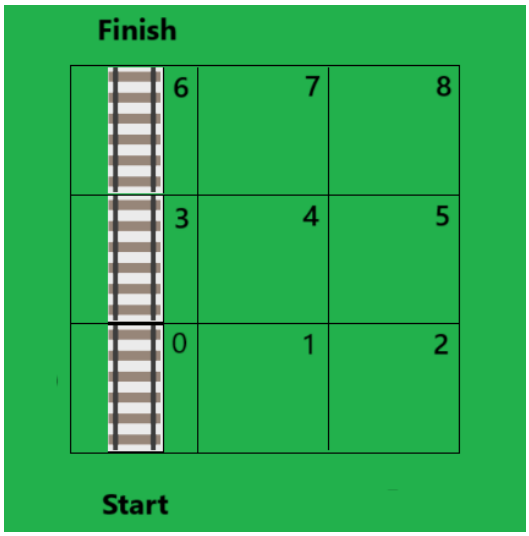
To me now it's still puzzling, since everything will happen instantly. So, I feel like introducing speed is important for comprehension (although it's written that speed may be ignored at this stage).

Intuitions on how to represent a train.

a. Represent the whole system as one object that takes the network, the train(s), the starting location etc. at initialization.

b. Represent the train and a network as two separate classes.

Successive indexing seems to simplify the task:



- We can use a 1d array now - as long as we know the correspondence between the index and the position (this would necessitate a drawing first every time - which is good for me).

- In asp, this would cancel the (x, y) out from the segment atoms:

```
segment(index, type)
```

In Python, this would be represented as a 1d dictionary the most easily:

```
Grid = {0: 32800,
      1: 0,
      2: 0,
      3: 32800,
      4: 0,
      5: 0,
      6: 32800,
      7: 0,
      8: 0}
```

Then what would movement on the grid look like?

The logic must be like this:

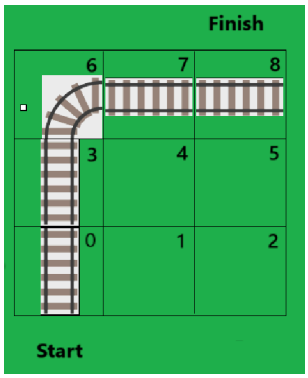
The starting point is known. We don't need the end point now. The train checks the type of segment it is on and moves accordingly. For simplicity, we can represent the train as a function instead of a class: it takes the network and returns the final destination.

```
def train(railroad, grid_size, start_segment):
    segment_action = {32800: "north"}
    total_length = len([x for x in railroad.values() if x != 0])
    visited_segments = [start_segment]
    current_segment = start_segment
    while visited_segments < total_length:
            current_segment = current_segment += grid_size[1]
    return current_segment
```

It seems that in a function like the one above I need a look up dictionary of tracks; for this simple case, I'll start with a lookup dictionary of just one type.

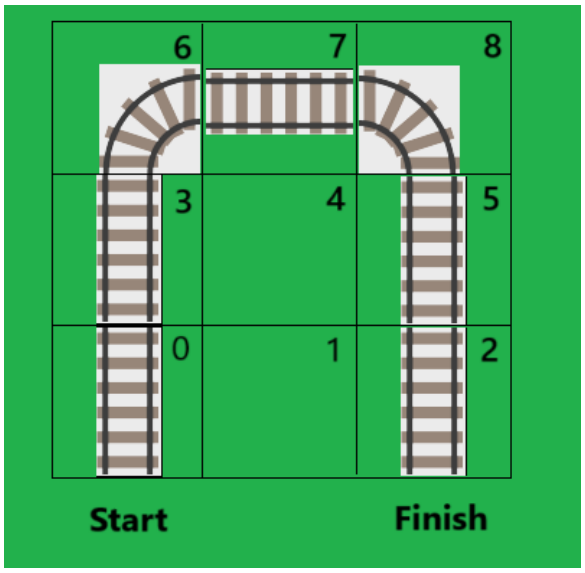
let's first make a really bad, unoptimized thing that just works;

Network 2: 1 Γ turn.



This works with my old code perfectly.

Network 3: П-turn



I will need to add one new type of rail: the east-south turn.

Another problem that will arise: my current setup only allows the movement by 32800 from south to north. To go the full П-turn, the movement from north to south must be allowed. Let's run in the existing state anyway to see what errors are raised (if any - potentially it can be just a wrong destination).

The naive solution: assign north heading and south heading segments different indexes.

Why doesn't it turn south?

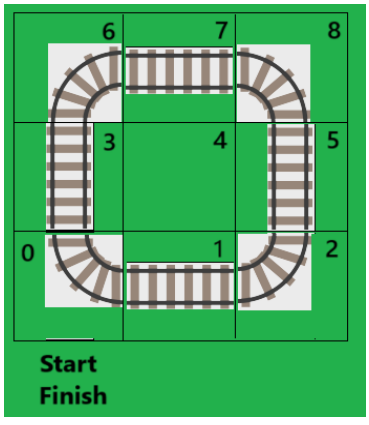
Yes, the naive solution worked. Now let's check generalization to previous solutions:

Yes, both previous works.

Question: do we need integer segment type indices at all? Isn't it simpler to encode segments as action? For non-choice railroads, it can be a simple string (my current basic case). For railroads with choice (max binary as it was said in the description) we can put all segments as tuples of possible actions). Simply put, the index seems to be an unnecessary - and confusing - intermediary.

However, for now let's stick to the current implementation with indexes and then try to throw them away.

Network 3: Round



I want the train to arrive at the same station it departs from (let it be a tourist railway).

The intuition now is that the current rule of counting the visited segments would stop the train one segment before the station.

NOTE: a train can start from a non-dead end segment (I may leave it for the later)

Let’s see how it runs.

Yes, it seems that straightforward indication of segment direction is better than having encoding as an intermediary.

What’s the next thing to do?

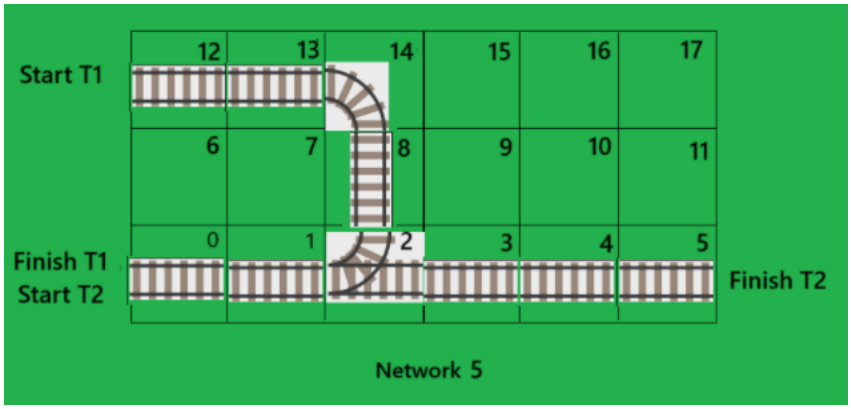
Okay, maybe encode a binary decision track? Let’s re-read the task to know if it’s expected at this stage.

Okay, there’s nothing said about the complexity of the track. All of the tracks that I encoded make collisions inherently impossible: if the trains move with the same speed, there is no collision.

So, the second train will only make sense for a more complex route than the one I have. Let’s draft such a route.

Question: should the collision avoidance mechanism be in the train (agent) or in the in environment? The intuition is that it must be in the environment.

I need to slightly change my “train” function to make it run on more complex tracks: besides “start”, it must also take “finish”.



There is a problem immediately: my current encoding with directions doesn’t work because the same segments are passed by different trains in different directions - so, the original encoding via indexes is more generalizable.

There are also **dead ends** - the last line of the table. I must use them at the ends of the network instead of normal straight segments.

NOTE: the inability for the trains to move backwards is key to encoding the railway network.

NOTE: the idea of moving till there are no unvisited segments doesn't hold any more - because the train must visit only the segments that lead to the destination.

NOTE: Maybe the idea of a **train-aware environment** is more effective than that of **environment-aware trains** (current state):

"No, trains do not directly control the switches that redirect them onto different tracks. Instead, a central operator, often a Train Control Officer (TCO) or a Signal Controller, uses electronic mechanisms to control the switches."

So, the **train-aware environment model adheres better to reality**.

IDEA: trains may be passed into the smart environment as a dictionary:

```
trains = {1: (start1, end1),
          2: (start2, end2)}
```

NOTE: when I remove "visited_segments" from the algorithm, the WHILE condition has no ground - instead of "visited_segments", I can use **the fact of arriving at the end destination**.

- First, it makes sense to test the new kind of train without a switch - on a fully deterministic route.

- Then, expand to a switch and think of the logic: it's interesting to explore how a real railroad works - **whether the train controls the switch or the environment controls the switch and thus the train**;

NOTE: dead ends also have two directions: **no movement** in case of arrival and **next segment** in case of departure!

"Tip: for your encoding, start as simple as possible. First work through having just a single agent move around -- then include a second agent. We're not looking for efficiency, just something that works. The idea is to work through the necessary steps and think through some of the challenges that such a paradigm presents. Don't worry about animating or visualizing the solution. We can ignore things like earliest departure time (this means the trains can leave whenever they want), speed, and malfunctions. You do need to stay on the tracks, you may not move backward, and you may not collide."

Then, there is a tip: start as simple as possible. Have one agent only. Then include the second. Don't worry about efficiency, just build a working small system. The intention now is just to dive into the problem. I should worry about animating or visualizing at this stage. I should neglect exact departure times: trains may leave at any time; I must also ignore the speed (different or

speed at all - just have “movement” instead) and malfunctions (later, perhaps, they’ll be encoded as probabilities). What I do need to do is:

- Stay on tracks;
- Ensure no backward moving;
- Ensure no collisions.

Exploring solutions from previous years:

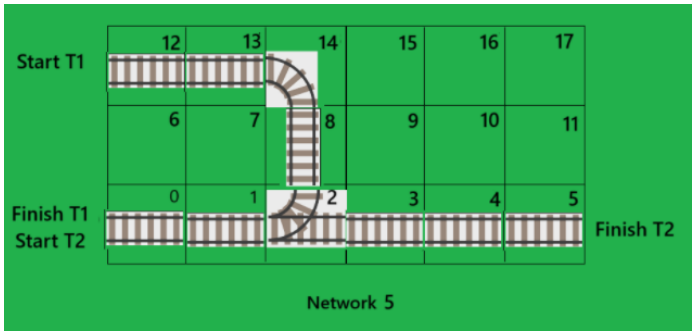
- Do the encodings have mandatory form?

Seems that no: they are individual per team, and we’re free to encode the environment as we want.

- Interesting examples of environments:

https://github.com/warfiaUni/RailwayScheduling/blob/main/data/environments/README.md

Encoding a simple switch environment:



First, I’ll just try to run one train on it: T1. Done. Both train 1 and train 2 run separately well. Now I need to figure out how to approach simultaneous runs / collision avoidance.

But before I proceed to collision avoidance, **let’s observe a collision!**

INTUITION: a collision is when two trains are on the same segment at the same moment and this segment is NOT a start / destination.

INTUITION: maybe it’s good to rename my “train” function into “control system”, for example. +

INTUITION: **have trains run on separate instances of the same railroad system, making them exchange message on the current positions** at each moment of time to avoid collisions;

Surprisingly, introducing a simple timer allows me to imitate parallelism of train runs effectively. It looks enough for now. What I need to do next is to introduce **collision alert**:

The simple logic of a collision:

If two or more trains are on the same segment at the same moment of time - it's a collision!

Later, I need to refine it by allowing simultaneous presence of several trains at end stations.

Basically, all I need is the report to find out if there was a collision. Done, now the report (I call it a "dispatcher") looks like this:

[(1, 1, 1), (1, 2, 2), (1, 3, 3), (1, 4, 4), (1, 5, 5), (2, 1, 13), (2, 2, 14), (2, 3, 8), (2, 4, 2), (2, 5, 1), (2, 6, 0)]

How do I formalize the condition for a collision looking at this report?

A collision is simply when there are two or more tuples whose first elements are different but whose second and third elements are the same. How do I introduce this in Python?

Naive: iterate over all tuples: fix the first, compare to every other (this seems very ineffective);

Less naive: use "Collections" (I feel like there must be a tool for this)

Hypothesis: there is an edge case of two trains "avoiding" collision by a margin of 1.

Well, with even length of the track opposite trains **do collide** - so, my "dispatcher" system works.

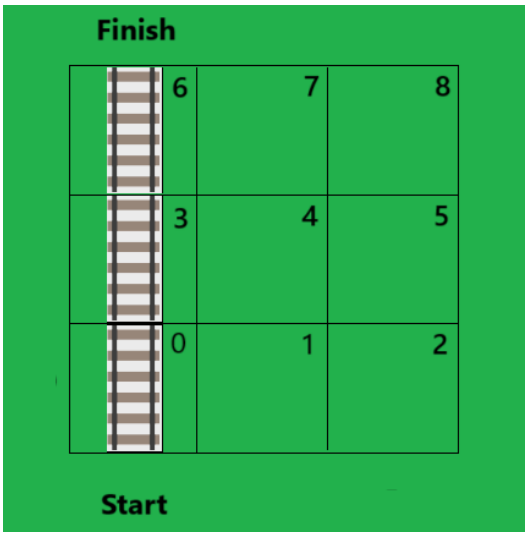
Intuition: to cover the edge case of opposite trains "avoiding" collision on an odd length track, the collision condition might be expanded to -1 / +1 to the current position of the second train at the same moment of time.

TRANSLATING TO ASP

How do I translate the same problem into ASP? The first intuition is that it's gonna all or mostly all be about *constraints* - so, what in my Python encoding was an instruction (do this and do that), in ASP must become a constraint (don't do this and don't do that).

First, anyway, I'll need to represent the environment. The way that I encoded the environment in Python is not so far from ASP anyway - while in Python it's an array, in ASP it will be a set of separate facts.

Let's take the simplest environment (railroad network) and try to represent it as ASP facts:



I'll create an atom named "segment". This atom will hold variables that:

- Describe the position of the segment on the grid;
- Describe the type of the segment;

So, it's two variables inside an atom:

```
segment(position, type)
```

Let's describe this simple route in ASP atoms:

```
segment(0, 128).
segment(1, 0).
segment(2, 9).
segment(3, 32800).
segment(4, 0).
segment(5, 0).
segment(6, 8192).
segment(7, 0).
segment(8, 0).
```

Good. My python encoding has shown that this is enough to represent a railroad and move the trains. Now let's plug this small encoding into Clingo simply to see if it causes any errors.

No, it doesn't. The answer set is just as the input:

Answer: 1
segment(0,128) segment(1,0) segment(2,9) segment(3,32800) segment(4,0) segment(5,0) segment(6,8192) segment(7,0) segment(8,0)

Now, what's the next step? The next step is to think about what a train must not do.

1. The train must stay on the tracks. How does it translate into constraints?

ASP constraint:

```
:- travelling(_, type), type == 0.
```

INTUITION: to fully describe the railroad, we need to also describe the departure and arrival points. For this, it's enough to introduce two atoms:

```
start(position).
end(position).
```

So, the full environment encoding must be like:

```
start(0).
end(6).
segment(0, 128).
segment(1, 0).
segment(2, 9).
segment(3, 32800).
segment(4, 0).
segment(5, 0).
segment(6, 8192).
segment(7, 0).
segment(8, 0).
```

The big question is: **what should the output be after all?**

INTUITION: it seems very counterintuitive to deal with successive actions in an environment that implies solving everything at once.

QUESTION: The "standard" approach in ASP modelling is **overgenerate → kill**. How do I (and should I) apply this to the railroad problem?

QUESTION: What should the output look like?

Guess 1: the output should look like the whole schedule of the train(s) being in a certain position at a certain moment of time. In this case, the overgeneration step will produce all possible combinations of train positions at different moments of time (this seems very expensive computationally, but may be improved using constraints).

To implement this, I need to introduce time into my encoding.

TASK: generate answer sets where the train will be in each possible position in each possible moment of time.

The output should consist of atoms that look like this:

```
travel(TRAIN, MOMENT, POSITION)
```

What makes up this atom?

`TRAIN` - should be an atom given in the input, holding a numeric … ;
`MOMENT` - should be a numeric variable; in what range? At this stage, we connect time directly to physical movement: each new movement - a new moment of time. This variable must be generated in range between 0 and maximum time that the train can travel on the given network; there are two approaches:

- Very naive: take the max time as the number of all cells in the grid;
- Less naive: take the max time as the number of passable cells in the grid, i.e. ones that are not 0 track type;

Let's try both. For the first one, in order to find "max_time", I'll simply count all positions in the grid ignoring their type (so, including the impassable positions as well).

The first one worked and now I can establish the range from which moments of time will be generated.

I can also generate moments correctly - now there are nine "moment(MOMENT)" atoms in the output.

`POSITION` - position is taken from the network description;

Now I have the output:

```
travel(1,1,0) travel(1,2,0) travel(1,3,0) travel(1,4,0) travel(1,5,0)
travel(1,6,0) travel(1,7,0) travel(1,8,0) travel(1,9,0) travel(1,1,1)
travel(1,2,1) travel(1,3,1) travel(1,4,1) travel(1,5,1) travel(1,6,1)
travel(1,7,1) travel(1,8,1) travel(1,9,1) travel(1,1,2) travel(1,2,2)
travel(1,3,2) travel(1,4,2) travel(1,5,2) travel(1,6,2) travel(1,7,2)
travel(1,8,2) travel(1,9,2) travel(1,1,3) travel(1,2,3) travel(1,3,3)
travel(1,4,3) travel(1,5,3) travel(1,6,3) travel(1,7,3) travel(1,8,3)
travel(1,9,3) travel(1,1,4) travel(1,2,4) travel(1,3,4) travel(1,4,4)
travel(1,5,4) travel(1,6,4) travel(1,7,4) travel(1,8,4) travel(1,9,4)
travel(1,1,5) travel(1,2,5) travel(1,3,5) travel(1,4,5) travel(1,5,5)
travel(1,6,5) travel(1,7,5) travel(1,8,5) travel(1,9,5) travel(1,1,6)
travel(1,2,6) travel(1,3,6) travel(1,4,6) travel(1,5,6) travel(1,6,6)
travel(1,7,6) travel(1,8,6) travel(1,9,6) travel(1,1,7) travel(1,2,7)
travel(1,3,7) travel(1,4,7) travel(1,5,7) travel(1,6,7) travel(1,7,7)
travel(1,8,7) travel(1,9,7) travel(1,1,8) travel(1,2,8) travel(1,3,8)
travel(1,4,8) travel(1,5,8) travel(1,6,8) travel(1,7,8) travel(1,8,8)
travel(1,9,8)
```

Trying to obtain the answer only for passable segments:

```
travel(1,1,0) travel(1,2,0) travel(1,3,0) travel(1,1,1) travel(1,2,1)
travel(1,3,1) travel(1,1,2) travel(1,2,2) travel(1,3,2) travel(1,1,3)
travel(1,2,3) travel(1,3,3) travel(1,1,4) travel(1,2,4) travel(1,3,4)
travel(1,1,5) travel(1,2,5) travel(1,3,5) travel(1,1,6) travel(1,2,6)
travel(1,3,6) travel(1,1,7) travel(1,2,7) travel(1,3,7) travel(1,1,8)
travel(1,2,8) travel(1,3,8)
```

The easiest way to understand what happens / debug is to output the “max_time” value for different selection conditions.

It works correctly. Let’s then look at the output more closely. Let’s temporarily remove the train for visibility.

INTUITION: I need a choice rule, not a normal one - it doesn’t generate all permutations of time / position.

20.04.25

TASK: rewrite “travel” atom generation as a choice rule.

Done. Now I have a choice:

- To keep developing the choice rule / what precedes it to make it more precise;
- Start writing the constraints to limit the output;

INTUITION: Do we need to encode 0 type tracks at all?

It seems that we could just leave it out, but for completeness of input it feels like 0 segments of the grid must be encoded too.

Good. Now I have just 84 answers. I can now play around with constraints:

Constraint Idea	**ASP Phrasing**
The train must stay on tracks	Implicitly satisfied by the generator choice rule.
The train must not go back	There must be no answer set where at a later moment in time the train occurs in an earlier position. **SOLVED.**
The train must not jump between segments	There must be no answer set where the train at the next moment of time is farther away from its

	previous position into the future than by 1 segment. No! This is not correct. Even in my very simple route “jumping” farther ahead than by 1 is inherent - simply due to jumps between rows of the grid when moving north. A naive approach: the jump must be no bigger than the width of the grid. In this case, I need to pass grid size as part of the input. With this naive assumption, the constraint can be phrased like this: There must be no answer set where the difference in positions reached within one time step is bigger than the width of the grid. **SOLVED.** Of course, it will cause some issues with more complex grids, but for now it’s enough.
The train must not be in two positions at the same moment in time.	There must be no answer set where the moment of time is the same but positions are different. **SOLVED.**
The train must start from the departure station and arrive at the destination station.	There must be no answer set where at the first moment of time the train is not at the starting point and at the last moment of time the train is not at the destination point. **SOLVED.**

NOTE: We don’t introduce a constraint that would prohibit the train to stay in a position for an amount of time - when there are two trains, one can wait for another.

INTUITION: The waiting time probably introduces the need for more time than the length of the grid. We’ll deal with it later.

Currently the output is:

```
Answer: 1
travel(1,0) travel(2,6) travel(3,6) - train jumps over a segment
Answer: 2
```

travel(1,3) travel(2,6) travel(3,6) - **train starts from a wrong position instead of departure station**
Answer: 3
travel(1,0) travel(2,3) travel(3,3) - **train gets stuck in the middle of the route**
Answer: 4
travel(1,0) travel(2,3) travel(3,6) - **train goes correctly**
Answer: 5
travel(1,3) travel(2,3) travel(3,3) - **train starts wrong and gets stuck immediately**
Answer: 6
travel(1,3) travel(2,3) travel(3,6) - **train starts wrong, gets stuck but arrives at the destination by luck**
Answer: 7
travel(1,0) travel(2,0) travel(3,3) - **train lingers at the start, goes to the middle and never reaches the destination**
Answer: 8
travel(1,0) travel(2,0) travel(3,6)- **train lingers at the start and then jumps to the destination**
Answer: 9
travel(1,0) travel(2,0) travel(3,0) - **train is stuck at the departure point**
Answer: 10
travel(1,6) travel(2,6) travel(3,6) - **train is stuck at the destination point**
SATISFIABLE

Let's analyse what happens in each case.

As a result of applying new constraints, I stayed with only three answers:

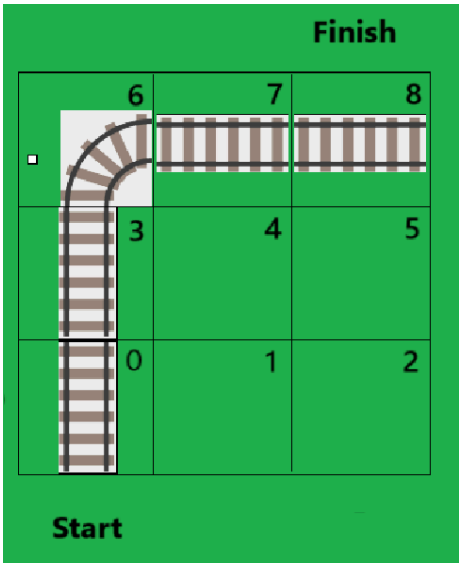
Answer: 1
travel(1,0) travel(2,3) travel(3,6)
Answer: 2
travel(1,0) travel(2,6) travel(3,6)
Answer: 3
travel(1,0) travel(2,0) travel(3,6)
SATISFIABLE

To leave only the right answer in place, I also need to deal with "jumping".

SOLVED.

TASK: try more complex deterministic single train environments.

Γ-turn:




```
Answer: 1
travel(1,0) travel(3,6) travel(4,6)
travel(5,8) travel(2,3) - jumping between
6 and 8, getting stuck at 6 and skipping
segment 7
Answer: 2
travel(1,0) travel(3,6) travel(4,7)
travel(5,8) travel(2,3) - correct trip
Answer: 3
travel(1,0) travel(3,6) travel(4,8)
travel(5,8) travel(2,3) - jumping between
6 and 8, getting stuck at the destination
Answer: 4
travel(1,0) travel(4,6) travel(5,8)
travel(2,3) travel(3,3) - stuck at 3,
jumps to the destination
Answer: 5
travel(1,0) travel(2,0) travel(4,6) travel(5,8) travel(3,3) -
SATISFIABLE - stuck at the beginning, jumps over segment 7
```

So, the constraint on never jumping over segments is crucial. Let's draft it again:

Constraint Idea	**ASP phrasing**
A train must never jump over segments. The intuition now is to distinguish between what we consider "jumping" based on the type of segment it happens from: South-north track - jump is step > grid_width in any direction. East-west track - jump is step > 1 in any direction. Problem: "TYPE" is lost in the 'travel' atom;	There must be no answer set where: ● The track type implies south-north or north-south movement but the absolute positional change is not equal to the width of the grid; ● The track type implies east-west or west-east movement but the absolute positional change is not equal to 1; **SOLVED (preliminary)**

Solution: “TYPE” can be extracted from segment atoms that map positions to types.	

The answer is actually not so bad: I forgot that answer atoms are not ordered. So I need to simply follow the moment of time variable to track movement.

Yes, it’s actually good - it contains the correct trip:

```
Answer: 2
travel(1,0) travel(3,6) travel(4,7) travel(5,8) travel(2,3)
```

Let’s analyse the problems with other answers and maybe come up with finer constraints.

П-turn:

INTUITION: At this stage, **visualising answer sets would be of great help**.

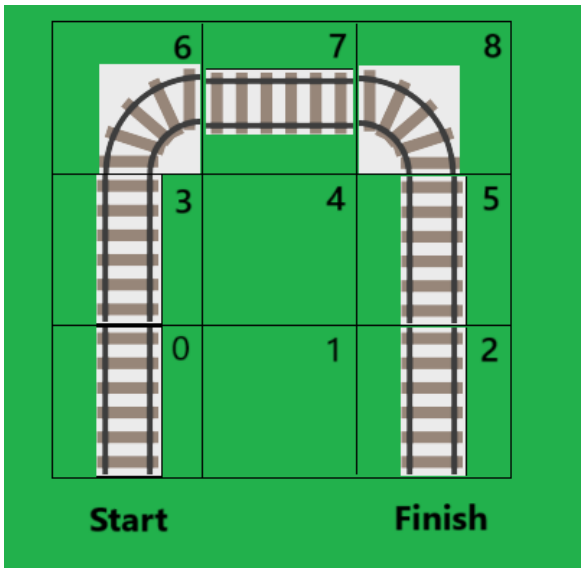
INTUITION: there must be an upper limit for trip - so, even if we allow the train to stop on track, the total travelling time must not exceed this limit - answers where the train doesn’t reach the destination within the allotted time must be eliminated (there actually **already are**).

Now for the 2nd deterministic one train environment the system outputs only the correct answer:

```
travel(2,3) travel(3,6) travel(4,7) travel(5,8) travel(1,0)
```

But as usual, I have a feeling that I might have overconstrained. Hence, let’s try more simple deterministic environments and see where it falters.

П-turn:



I can directly take encoding from Python, only changing formatting slightly.

It’s unsatisfiable. Why?

H1: because of the constraints on movement; currently the south-north constraint doesn’t allow moving south;

Testing: is there a concept of absolute value in ASP?

Answer: no;

So, the problem is clearly in moving south. Which segments - among the ones used in the Π-turn network - assume that the train must be moving south?

The ones in positions 8, 5, 2. Let's check their indexes:

```
segment(2, 128).
segment(5, 32800).
segment(8, 4608).
```

Let's check how these segments are described in my encoding.

4608 seems to be missing. Let's check again. Yes, it is missing. How should I describe it in terms of movement?

If entered from the south, 4608 ensures transition by 1 to the west.

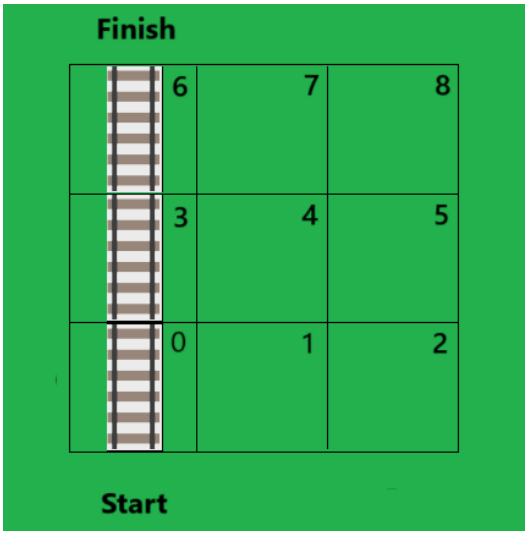
If entered from the west, 4608 ensures transition by grid width to the south.

In this exact case of the Π-turn, 4608 is only entered from west, so the legal transition is only by 1 to the south.

Let's take a different approach and first learn to send the train backwards by every segment.

So, now **the task is:**

Send the train by the first deterministic single train route ('straight_line') from north to south.



Naively introducing the opposite constraint ('no answer set with next moment position different from current minus grid width).

QUESTION: How do we introduce constraints in ASP so that EITHER of them applies, but not two together?

Surprisingly, we can integrate choice rules right into the constraint's body:

```
:- grid_size(HEIGHT, WIDTH),
   segment(POSITION_1, (128; 32800; 8192)),
   travel(MOMENT, POSITION_1),
   travel(MOMENT+1, POSITION_2),
   not 1 {POSITION_2 = POSITION_1 + WIDTH;
          POSITION_2 = POSITION_1 - WIDTH} 1.
```

But the train is still sent in the same direction: from south to north. Let's **send it in reverse**:

- First, I need to simply swap values in the "start" and "end" atoms.

The whole thing immediately becomes unsatisfiable. Why?

H1: because of how the types of tracks are described in the constraint. -

Testing H1:

- Let's switch the type-direction constraint off and see what happens.

Still unsatisfiable! So, it might not be about this constraint! Then what?

H2: because of the constraint on the order of moments / positions! +

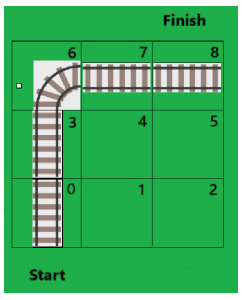
```
:- travel(MOMENT_1, POSITION_1),
   travel(MOMENT_2, POSITION_2),
   MOMENT_1 < MOMENT_2,
   POSITION_1 > POSITION_2.
```

It basically prohibits moving back.

Yes, switching it off allows travelling by decreasing numeric positions ("backwards" is not a correct word).

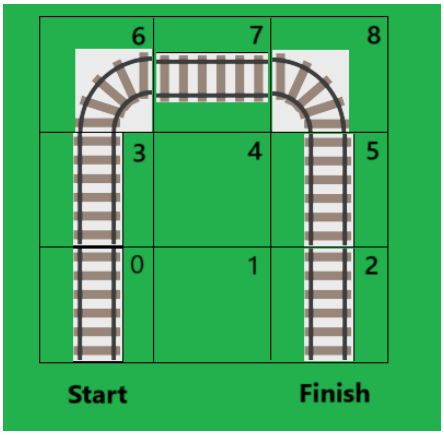
Okay, let's encode the reverse Γ-turn. But isn't there something simpler?

Which segments must be considered in the constraint? The ones from the Γ-turn that imply east-west / west-east movement.



SOLVED.

Let’s try a more complex track of Π-shape in the “normal” direction first:



Are there any segments in this track that are missing the current encoding?

Yes, the segment number 8 isn’t encoded.

Works!

Let’s run the train on the same track in reverse.

Works again!

The current encoding allows to run single route deterministic routes of any complexity - perhaps.

Let’s build more complex routes for one train in order to cover as more segment types:

Notes from the class:

NO SWITCH => NO ACTION CHOICE

If there is a diverging switch (*only* right or left), it’s a good idea to assign the default action (`if action not in [right, left], action = right`).

Malfunctions: randomly assigned.

How do we randomize in ASP? No way, it’s actually imported from the Python wrapper.

A malfunction has two parameters: train and duration.

We encode speed as inverse: hours per mile instead of miles per hour.

POTENTIAL DIRECTIONS:

- Introducing acceleration into simulation.
- Inclination of the track

We must use a **common fact format** instead of different custom encodings.

Arrival time must be a soft constraint - we may ignore it.

The output format is strict.